LLM API-Only Evaluation

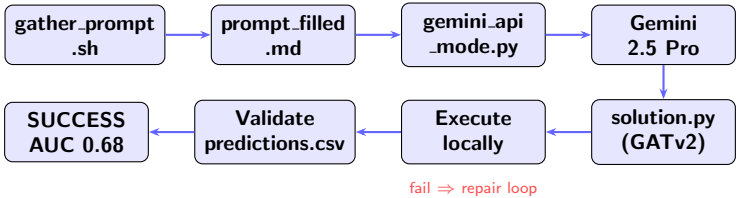
Gemini 2.5 Pro · GNN Link Prediction

GNN Rising Star

May 13,2026

GNN code generation using Gemini 2.5 Pro API
with a prompt-fill + execute-and-repair loop

Pipeline Overview



Key principle: API-only

- LLM generates code via API call
- Code runs *locally* (no agent shell)
- Traceback fed back on failure

Competition: NetLinkArena

- Link prediction, citation graph
- 3,327 nodes, 2,742 features
- Metric: AUC-ROC

Step 1 — gather_prompt_data.sh

Script content:

```
#!/bin/bash
echo "=== SECTION 1: README ==="
cat README.md

echo "=== SECTION 2: REPO TREE ==="
find . -type f -not -path "./.git/*"

echo "=== SECTION 3: DATA SAMPLE ==="
python3 -c "
import pandas as pd
df = pd.read_csv('data/public/train_edges.csv')
print('shape:', df.shape)
print(df.head(5).to_string())
# ... repeat for all CSVs
"

echo "=== SECTION 4: SUBMISSION FORMAT ==="
grep -A 30 'How to Submit' README.md

echo "=== SECTION 5: REQUIREMENTS ==="
cat requirements.txt
```

How to run:

```
cd NetLinkArena
bash gather_prompt_data.sh \
  > prompt_filled.md
cp prompt_filled.md \
  ../api_gemini/prompt_filled.md
```

5 sections collected:
1. README (competition description)
2. Repo file tree (find output)
3. CSV shapes + head(5) per file
4. Required submission format
5. requirements.txt + path notes

Same process as CGCC —

prompt_filled.md — What the LLM Receives

System instruction (fixed for all competitions):

```
You are solving a GNN coding competition.
Respond in EXACTLY this format:

<plan>
5-10 bullet plan: features, model
architecture, training, validation,
submission file format.
</plan>

<code>
Complete runnable Python script.
- Use only listed libraries
- Run as: python solution.py
- Set seeds for reproducibility
- CPU only, within 60 minutes
- Write predictions.csv exactly
</code>
```

User prompt (slots filled by .sh):

- **Section 1: README**
- **Section 2: Repo Tree**
- **Section 3: Data Sample**
- **Section 4: Submission Format**
- **Section 5: Requirements**

Key constraint added:
y_pred must be continuous probability in $[0, 1]$ — NOT binary (required for AUC-ROC scoring)

gemini_api_mode.py — The Orchestrator

Configuration:

```
from google import genai
from google.genai import types

MODEL         = "gemini-2.5-pro"
TEMPERATURE   = 0
TOP_P         = 0.99
MAX_TOKENS    = 10000
REPAIR_LIMIT  = 5

ARENA_DIR   = SCRIPT_DIR.parent / "NetLinkArena"
PROMPT_PATH= SCRIPT_DIR / "prompt_filled.md"
RUN_DIR     = SCRIPT_DIR / "run_gemini"
```

API call (new google-genai SDK):

```
client = genai.Client(
    api_key=os.environ["GOOGLE_API_KEY"])
config = types.GenerateContentConfig(
    max_output_tokens=MAX_TOKENS,
    temperature=TEMPERATURE, top_p=TOP_P)
response = client.models.generate_content(
    model=MODEL, contents=messages,
    config=config)
```

Message format:

```
# Role: 'user' or 'model' (not 'assistant')
types.Content(
  role="user",
  parts=[types.Part(text="...")]
)
```

SDK note:
Old (deprecated): `google.generativeai`
New: `from google import genai`
Install: `pip install google-genai`

Per-attempt artifacts saved:

attempt_N_raw_response.txt
attempt_N_plan.md
attempt_N_solution.py
attempt_N_execution_log.txt
attempt_N_validation_log.txt

Step 2 — Get & Set the API Key

1. Get the key:

Go to: aistudio.google.com/api-keys
→ Create API key
→ Select your project
→ Copy the key

2. Export in terminal:

```
export GOOGLE_API_KEY=your_key_here
# Verify it is set:
echo $GOOGLE_API_KEY
```

3. Install SDK:

```
pip install google-genai
```

Model availability:

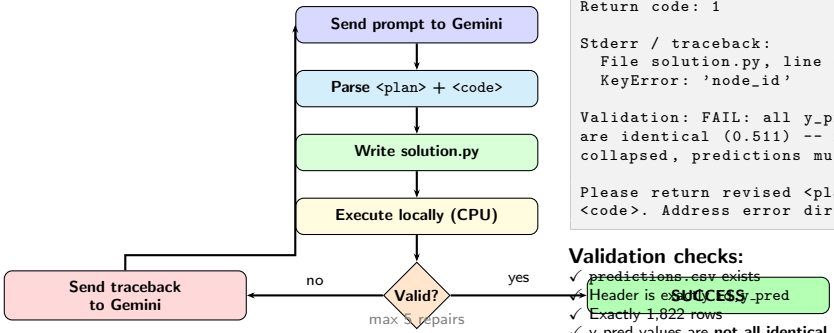
Model	Free	Paid
gemini-2.5-pro	No	Yes
gemini-2.5-flash	Yes	Yes
gemini-2.0-flash	Yes	Yes

Common errors:

× `gemini-3-pro` — does NOT exist
× 429 error — enable billing

`gemini-2.5-pro` requires payment.
$10 credit is sufficient for a full run.

The Execute-and-Repair Loop ($k \leq 5$)



Repair message sent to Gemini:

```
"The script you produced failed.

Return code: 1

Stderr / traceback:
  File solution.py, line 42
  KeyError: 'node_id'

Validation: FAIL: all y_pred values
are identical (0.511) -- model
collapsed, predictions must vary.

Please return revised <plan> and
<code>. Address error directly."
```

Validation checks:

- ✓ ~~predictions.csv exists~~
- ✓ Header is exactly ~~node_id,y_pred~~
- ✓ ~~Exactly 1,822 rows~~
- ✓ y_pred values are **not all identical**

Step 3 — Run the Full Pipeline

Full command sequence:

```
# 1. Fill the prompt
cd NetLinkArena
bash gather_prompt_data.sh \
  > prompt_filled.md
cp prompt_filled.md \
  ../api_gemini/prompt_filled.md

# 2. Set API key
export GOOGLE_API_KEY=your_key_here

# 3. Run
cd ../api_gemini
python gemini_api_mode.py
```

Terminal output:

```
=== Gemini attempt 0 | gemini-2.5-pro
Parse error: Missing <plan> block

=== Gemini attempt 1 | gemini-2.5-pro
Running solution.py from NetLinkArena
Epoch 001: Loss:1.52, Val AUC:0.500
Epoch 006: Loss:2.97, Val AUC:0.681 (best)
Validation: OK 1822 rows id,y_pred
Success: predictions written correctly.
Total: 130.9s | repairs=1
```

Artifacts in `run_gemini/`:

config.json
environment.txt
prompt_filled.md (snapshot)
attempt_0_raw_response.txt
attempt_0_parse_error.txt
attempt_1_plan.md
attempt_1_solution.py
attempt_1_execution_log.txt
attempt_1_validation_log.txt
final_solution.py
predictions.csv
run_summary.csv

Results — Gemini 2.5 Pro on NetLinkArena

Run summary:

Parameter	Value
LLM model	gemini-2.5-pro
Provider	Google AI Studio
Temperature	0
Repairs used	1
Total time	130.9 seconds
Success	Yes

Generated GNN:

Architecture	GATv2 (2 layers)
Dims	2742→32×8h→64
Decoder	Dot product + sigmoid
Optimizer	Adam, lr=0.005
Early stopping	patience=30

Performance vs baselines:

Summary — API-Only LLM Evaluation

What we built:

1. `gather_prompt_data.sh`
2. `prompt_filled.md` (5 sections)
3. `gemini_api_mode.py`
4. Execute-and-repair loop ($k \leq 5$)
5. Validation with diversity check

Key design choices:

- Temperature=0 — deterministic code generation
- Max 5 repair iterations
- Validation catches collapsed models
- All artifacts saved for reproducibility

How to reproduce:

1. Clone NetLinkArena repo
2. `bash gather_prompt_data.sh > prompt_filled.md`
3. `cp prompt_filled.md ../api_gemini/`
4. `export GOOGLE_API_KEY=...`
5. `python gemini_api_mode.py`

Only the API key differs between runs.
The prompt template is fixed and frozen.

Final Result:

Gemini 2.5 Pro → GATv2
Val AUC-ROC = 0.6817
1 repair · 130.9s · CPU only